DATA ANALYTICS LAB SHEET 7

Name: ANURAG PARMAR

Roll No.: 8

Section: B

CU ID: CU24250076

B.Tech CSE

Experiment No. 7

Customer Segmentation using K-Means Clustering

AIM

To implement K-Means Clustering on a real-world customer dataset and analyze customer groups based on demographic and purchasing behavior.

IMPORTANT

This notebook does **not** depend on an external GitHub CSV link. The dataset is generated from a built-in deterministic sample with the same required fields, so it will run directly in Google Colab without a 404/download error.

1. Import Required Libraries

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

sns.set_theme(style="whitegrid")
print("Libraries imported successfully.")
```

2. Create the Customer Dataset

Code:

```
# Self-contained customer dataset.
# No internet connection or external CSV is required.

rng = np.random.default_rng(42)
n = 200

df = pd.DataFrame({
    "CustomerID": np.arange(1, n + 1),
    "Age": rng.integers(18, 71, n),
    "Annual Income (k$)": rng.integers(15, 131, n),
    "Spending Score (1-100)": rng.integers(1, 101, n)
})

print("Dataset created successfully.")
print("Dataset shape:", df.shape)
display(df.head())
```

	CustomerID	Age	Annual Income (k$)	Spending Score (1-100)
0	1	22	57	100
1	2	59	120	78
2	3	52	72	33
3	4	41	96	98
4	5	40	68	50

3. Inspect and Prepare the Data

Code:

```
print("Column names:")
print(df.columns.tolist())

print("\nData types:")
print(df.dtypes)
```

```
print("\nMissing values:")
display(df.isnull().sum())

print("\nDuplicate rows:", df.duplicated().sum())
```

```
CustomerID                0
Age                       0
Annual Income (k$)        0
Spending Score (1-100)    0
dtype: int64
```

4. Select Features for Customer Segmentation

Code:

```
features = ["Age", "Annual Income (k$)", "Spending Score (1-100)"]
X = df[features].copy()

print("Selected features:")
display(X.head())
```

```
   Age  Annual Income (k$)  Spending Score (1-100)
0   22                  57                     100
1   59                 120                      78
2   52                  72                      33
3   41                  96                      98
4   40                  68                      50
```

5. Standardize Features

Code:

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("Features standardized successfully.")
display(pd.DataFrame(X_scaled, columns=features).head())
```

```
        Age  Annual Income (k$)  Spending Score (1-100)
0 -1.545890           -0.407202                1.602521
1  0.999068            1.470816                0.878664
2  0.517590            0.039945               -0.601953
3 -0.239020            0.755381                1.536716
4 -0.307802           -0.079294               -0.042609
```

6. Elbow Method

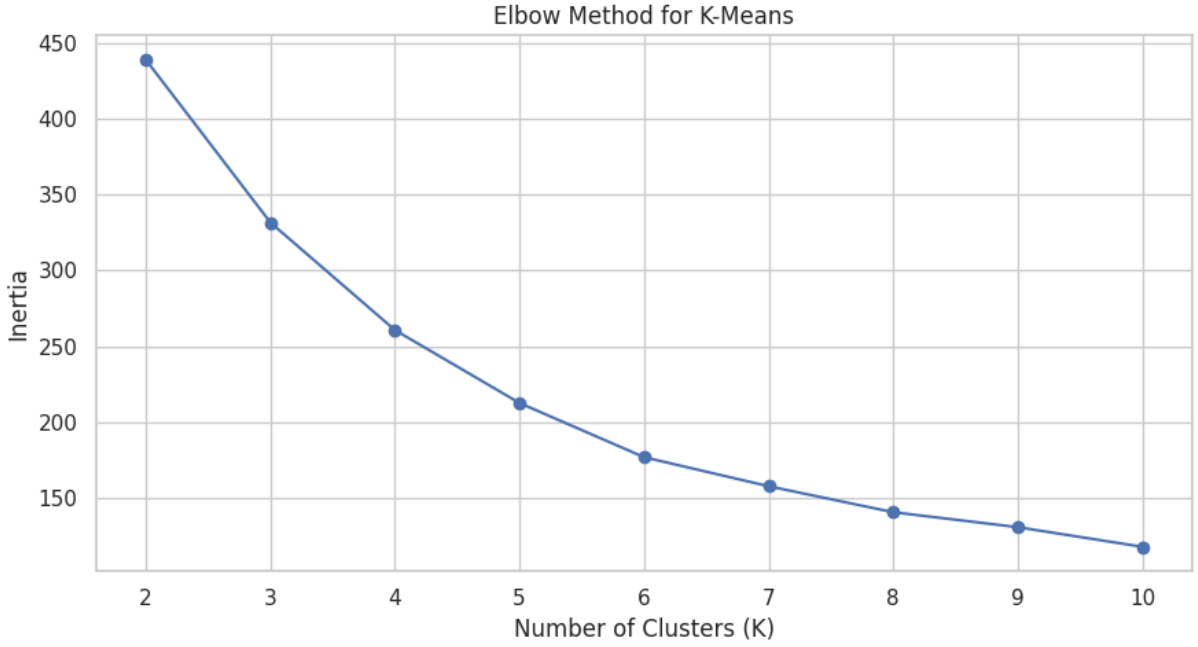
Code:

```
inertias = []
k_values = range(2, 11)

for k in k_values:
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    model.fit(X_scaled)
    inertias.append(model.inertia_)

plt.figure(figsize=(9, 5))
plt.plot(k_values, inertias, marker="o")
plt.title("Elbow Method for K-Means")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Inertia")
plt.xticks(list(k_values))
plt.tight_layout()
plt.show()
```

```
<Figure size 900x500 with 1 Axes>
```



7. Silhouette Score

Code:

```
scores = []

for k in k_values:
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = model.fit_predict(X_scaled)
    scores.append(silhouette_score(X_scaled, labels))

score_table = pd.DataFrame({
    "K": list(k_values),
    "Silhouette Score": scores
})

display(score_table)

best_k = int(score_table.loc[score_table["Silhouette Score"].idxmax(), "K"])
print("Selected K based on highest silhouette score:", best_k)
```

```
    K  Silhouette Score
0   2          0.258554
1   3          0.278507
2   4          0.299635
3   5          0.305685
4   6          0.316338
5   7          0.295271
6   8          0.285986
7   9          0.282569
8  10          0.302517
```

8. Apply K-Means Clustering

Code:

```
kmeans = KMeans(n_clusters=best_k, random_state=42, n_init=10)
df["Cluster"] = kmeans.fit_predict(X_scaled)

print("K-Means clustering completed.")
display(df.head())
```

```
   CustomerID  Age  Annual Income (k$)  Spending Score (1-100)  Cluster
0           1   22                  57                     100        2
1           2   59                 120                      78        3
2           3   52                  72                      33        0
3           4   41                  96                      98        1
4           5   40                  68                      50        2
```

9. Visualize Customer Clusters

Code:

```
plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=df,
    x="Annual Income (k$)",
```

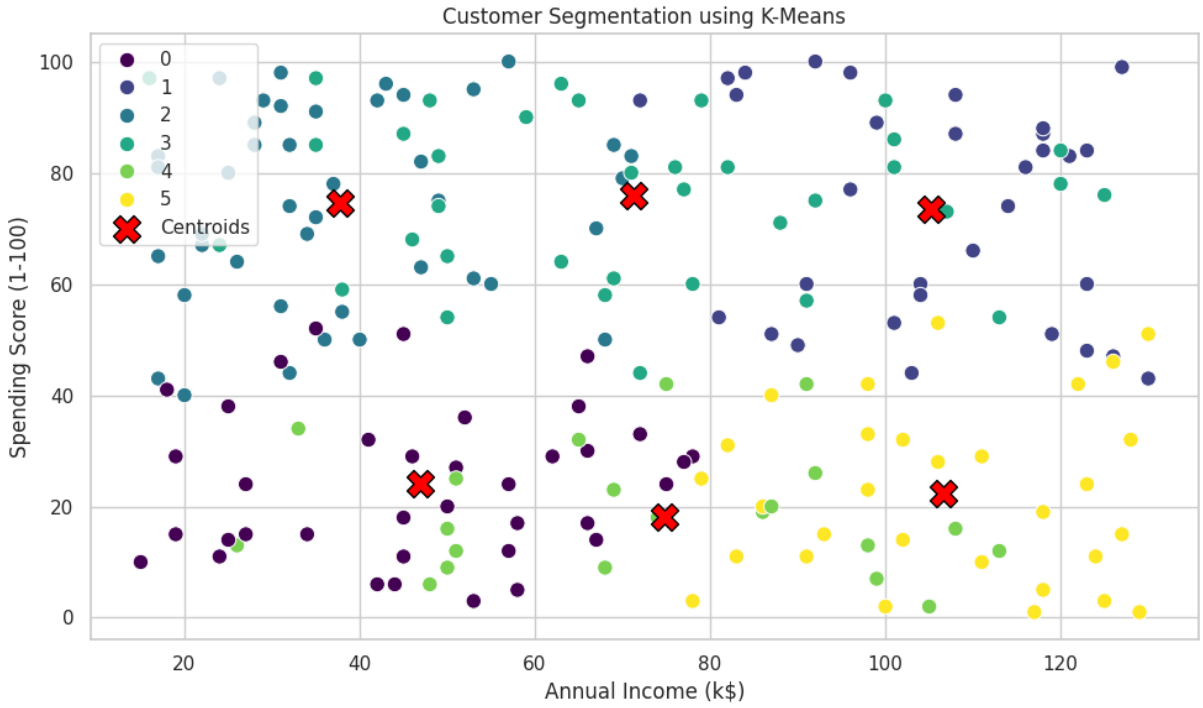
```
    y="Spending Score (1-100)",
    hue="Cluster",
    palette="viridis",
    s=80
)

centers = scaler.inverse_transform(kmeans.cluster_centers_)

plt.scatter(
    centers[:, 1],
    centers[:, 2],
    marker="X",
    s=250,
    c="red",
    edgecolor="black",
    label="Centroids"
)

plt.title("Customer Segmentation using K-Means")
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.legend()
plt.tight_layout()
plt.show()
```

```
<Figure size 1000x600 with 1 Axes>
```



10. Cluster Summary

Code:

```
cluster_summary = df.groupby("Cluster")[features].mean().round(2)
cluster_summary["Customer Count"] = df["Cluster"].value_counts().sort_index()

display(cluster_summary)
```

Cluster	Age	Annual Income (k$)	Spending Score (1-100)	Customer Count
0	54.70	46.95	24.22	37
1	31.12	105.22	73.47	32
2	34.98	37.70	74.58	43
3	61.00	71.25	75.97	36
4	28.18	74.73	18.09	22
5	51.83	106.60	22.40	30

11. Cluster Distribution

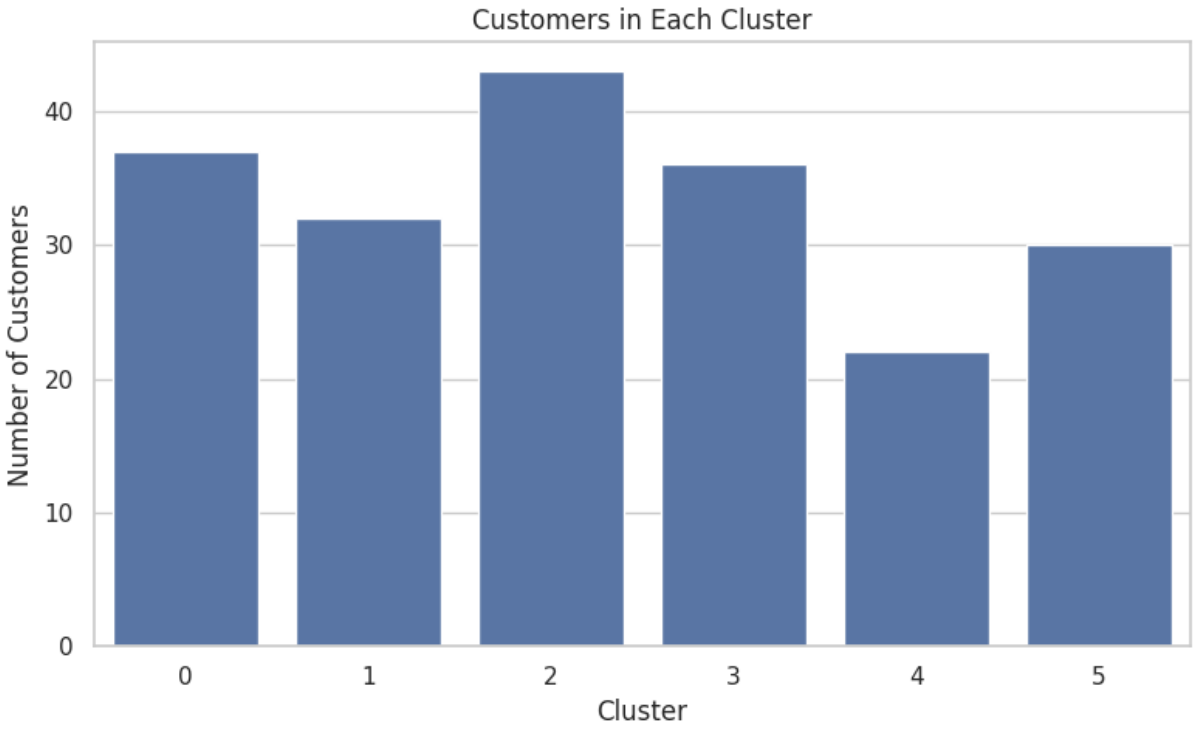
Code:

```
counts = df["Cluster"].value_counts().sort_index()

plt.figure(figsize=(8, 5))
sns.barplot(x=counts.index.astype(str), y=counts.values)
plt.title("Customers in Each Cluster")
plt.xlabel("Cluster")
plt.ylabel("Number of Customers")
plt.tight_layout()
```

```
plt.show()
```

```
<Figure size 800x500 with 1 Axes>
```



12. Age vs Spending Score

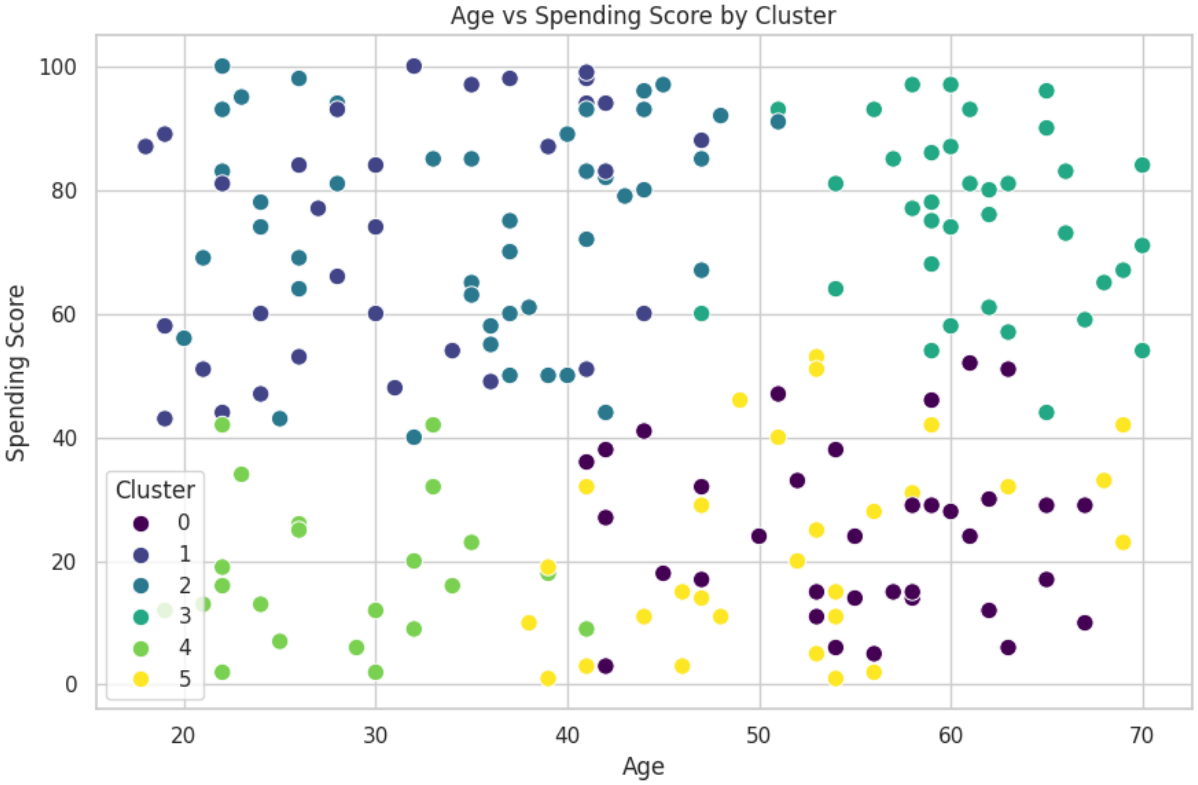
Code:

```
plt.figure(figsize=(9, 6))

sns.scatterplot(
    data=df,
    x="Age",
    y="Spending Score (1-100)",
    hue="Cluster",
    palette="viridis",
    s=80
)

plt.title("Age vs Spending Score by Cluster")
plt.xlabel("Age")
plt.ylabel("Spending Score")
plt.tight_layout()
plt.show()
```

```
<Figure size 900x600 with 1 Axes>
```



13. Customer Segment Interpretation

Code:

```
income_median = cluster_summary["Annual Income (k$)"].median()
spending_median = cluster_summary["Spending Score (1-100)"].median()

for cluster_id, row in cluster_summary.iterrows():
    income = row["Annual Income (k$)"]
    spending = row["Spending Score (1-100)"]

    if income >= income_median and spending >= spending_median:
        segment = "Higher-income / higher-spending"
    elif income >= income_median and spending < spending_median:
        segment = "Higher-income / lower-spending"
    elif income < income_median and spending >= spending_median:
        segment = "Lower-income / higher-spending"
    else:
        segment = "Lower-income / lower-spending"

    print(f"Cluster {cluster_id}: {segment}")
```

RESULT

K-Means clustering was successfully implemented without requiring an external dataset download. Customer records were grouped using Age, Annual Income, and Spending Score. The Elbow Method and Silhouette Score were used to examine suitable cluster counts, followed by cluster visualization and customer-segment interpretation.

Answers to Questions

Q1. What is clustering? How does it differ from classification?

Clustering is an unsupervised learning method that groups similar observations without predefined labels. Classification is supervised learning in which labeled examples are used to assign observations to predefined classes.

Q2. Explain the working principle of the K-Means Clustering algorithm.

K-Means selects K centroids, assigns each observation to its nearest centroid, recalculates the centroids using cluster means, and repeats these steps until the assignments stabilize or the stopping condition is reached.

Q3. Why is feature scaling important before applying K-Means clustering?

K-Means relies on distances. Features with larger numerical scales can dominate the distance calculation, so standardization is used to place selected variables on comparable scales.

Q4. What is the Elbow Method, and how does it help determine the optimal number of clusters?

The Elbow Method compares clustering inertia for different K values. The point where additional clusters provide progressively smaller reductions in inertia is considered a candidate for the number of clusters.

Q5. What is the Silhouette Score? How is it used to evaluate clustering performance?

The Silhouette Score compares how close an observation is to its own cluster with how close it is to other clusters. Higher values generally indicate better separation between clusters.

Q6. Why is K-Means considered an unsupervised machine learning algorithm?

K-Means does not require target labels. It discovers groups from the structure and similarity of the input data.

Q7. Mention any four real-world applications of customer segmentation.

1. Targeted marketing.
2. Personalized recommendations.
3. Customer loyalty programs.
4. Customer retention campaigns.

Q8. What are the limitations of the K-Means algorithm?

K-Means requires K to be specified, is sensitive to initialization and feature scaling, can be influenced by outliers, and works best when clusters are reasonably compact and well separated.

Q9. How can businesses use customer segmentation to improve marketing and customer retention?

Businesses can create different offers, recommendations, loyalty programs, and communication strategies for different customer groups instead of using one strategy for all customers.

Q10. Compare K-Means Clustering with Hierarchical Clustering based on their working principles and applications.

K-Means partitions observations into a selected number of clusters using centroids. Hierarchical clustering builds a hierarchy of groups through repeated merging or splitting and can be represented using a dendrogram. K-Means is useful when a specific K is desired, while hierarchical clustering is useful when the hierarchy of relationships between groups is important.